Distributed live stacking: follow-up items

For the author's review, morning of 2026-10-23

Where things stand

Every code task is merged and pushed. The task list is at 71 of 77. The six open tasks are your manual runs on the home network, set out step by step in `STARFRONT_DISTRIBUTED_STACKING_CHECKLIST.pdf` beside this file.

Repository	Branch and commit	CI on that commit
Starfront	`main` (this document's commit)	Python checks pass on Ubuntu and Windows (green since `5a6b5ce`, the first green run since the workflow was added)
Rapid Stacker Collab	`master` `96e7157`	every workflow passes: worker-ci (all three OSes and both hub end-to-end jobs), backend-ci, contract-ci, local-e2e, plugin-ci, site-ci

Collab's contract is now **1.29.0** (one additive change, below). A hub refuses workers older than its own contract, which only matters once worker builds are released: release the hub and workers together.

Nothing was deployed, released or signed. No AWS resource was touched.

Do first this morning

1. ~~Install Starfront's Python requirements on the Mac.~~ Done (you ran the pip install). Every Starfront check was re-run under your `python3` afterwards and passes, apart from the known cases in the checklist's "known gaps".
2. **Start the checklist** (`STARFRONT_DISTRIBUTED_STACKING_CHECKLIST.pdf`), Part 0 then Part 1. Part 2 (`local_night.py`) has already been run once with real frames, below, so expect the same result.

Decisions for you

Each is small. My recommendation comes first in each row.

#	Item	What it means today	Recommendation
1	**The release signing key**	A placeholder. Every desktop install and every worker self-update is refused by design. T507 cannot pass without it.	Generate it (`RSC/docs/runbooks/releasing-workers.md` §2), keep the private half offline, and paste the public half into `volunteer.py` and `self_update.rs`. Only you can do this.
2	**The hub's mirror of the worker manifest** (plan 4.6)	Never became a task, so it was never built. Volunteers install from the Starfront server instead of the hub. A LAN with no internet cannot install volunteers.	Build it: a small hub task (fetch `latest.json` and `.sig` hourly, serve them byte for byte at `/hub/v1/worker-manifest`). The desktop already tries it first.
3	**A desktop field for a collaboration's Discord webhook**	The server stores and uses `discordWebhookUrl`, but no screen sets it. Today only `local_night.py --discord-webhook` or the API does.	Add an owner-only field to the Collab tab's collaboration form.
4	**`stackContract` in the Starfront server's health** (plan 7.3)	Not built. Nothing reports which contract the server's front door speaks.	Add it; a few lines and a check case.
5	**Deleting a cloud-bound collaboration**	Collab's project delete and restore routes accept only a signed-in web session, so Starfront can only *pause* the cloud project.	Either accept pausing, or add a token-scoped owner delete to Collab's API (a contract change, then an AWS deploy, which is yours). Accepting is the cheaper choice until a real cloud collaboration needs deleting.
6	**Purge timing for deleted hub projects**	The purge follows the seven-day window by up to a day, because retention runs daily. Documented in the hub runbook.	Accept it; it errs on the side of keeping data.
7	**T310's wording**	It says "standalone hub", but projects now only come from Starfront (your direction). The checklist runs it through the local server.	Let me reword the task to match.

The real-frame run

`local_night.py` was run against eight of your Elephant's Trunk Ha lights (UC131, ASI2600MM Duo, 300 s) with the real XPSD folder: local server, hub and one Mac worker, frames uploaded with `collab-uploader`.

- **All 8 accepted**, 2 400 s of integration, `status.json` and the previews pushed to the server and shown on `/stack/p/<slug>`.
- **One real bug: the first frame waited about 10 minutes.** The first light fixes the grid's rotation, which moves the grid from revision 0 to 1. A prepare that had started on revision 0 asked the hub for that revision's catalogue, got the same "not found" answer as for a missing catalogue, built the old grid's catalogue locally, failed, and its message waited for the hub's 5-minute sweep to resend it. AWS has the same race.
- **Fixed** (Collab pull request #23, merged as `96e7157`, contract 1.29.0): the hub's "not found" now carries the current `grid_rev`; the worker re-reads the project and asks again within the same delivery, keeps one catalogue file per grid, and the engine reports a grid change before a frame is claimed rather than failing. Re-run on the same lights: the same race happened and was handled, the first frame was solved 11 s after upload, and all eight were accumulated within 16 s. Regression tests fail on the old code.

- Side effects to know: each worker re-fetches a project's catalogue once (the cache file name changed), and old per-grid catalogue files are not pruned from a worker's scratch (a grid moves rarely, so only a few build up).

What was found and fixed overnight

You may want to know about these before you test. The first would have broken the deployed server.

- **Found 2026-10-24, while building the three-machine lab: the LAN short-cut refused every telescope.** The server signed grants with the telescope owner's Starfront id in `sub`, while the hub keys members by collab user id and has no way to map one to the other. Uploads through the server's front door worked (it maps the id), but anything sent straight to the hub (the plugin's LAN short-cut, `collab-uploader` against a hub) was "not an active member of this project". Grants now carry the collab user id (`server/stack/auth.py` `collab_identity`); grants minted before the fix still work at the front door until they expire. Checked by `check_server.py` and by `tools/lab_setup.py smoke --via hub`.
- **The Starfront server refused every hub push.** `python-multipart` was in neither `requirements.txt` nor `server/deploy/install.sh`, so `request.form()` failed and the server answered 400. No verdict, preview or master could ever reach the pages. Fixed in `28ac374`; **the next server deploy installs it** (install.sh's pip line now includes it).
- **Part uploads were corrupted on Windows.** The server wrote parts in text mode, which turns every newline byte into two. Fixed in `9d0b6e4` (binary mode; no change on Linux, where the server runs).
- **CI was red in both repositories for most of the build**, and I had been watching only the plugin build. Starfront's Python checks had never passed. Collab's worker, backend and contract workflows broke during the build. All are fixed:
    - real Windows bugs in the hub and worker (free disk space unreadable on Windows; a directory under `/public` answered 500 instead of 404; macOS and Linux service files written with Windows separators);
    - one worker setting missing from the Terraform name list;
    - the contract check now builds the hub on the runner;
    - the hub end-to-end script gives its workers a scratch cap, so GitHub's small runners pass the 20 GB rule. The rule for real volunteers is unchanged.
- **`local_night.py` could not take real frames.** It now always joins its telescope, saves the 24-hour grant, prints the uploader commands, and takes `--region` and `--discord-webhook`. It also refuses to start without the server's requirements.
- **The uploader needs `projects` before `upload`.** It refuses an upload with "You have not joined a project yet" until `collab-uploader projects` has run once; `local_night.py`'s banner now prints the three commands in order.
- **`check_coordinator.py` had been crashing**, hidden by CI's allow-list, because it still expected a pasted join code to be stored. It now checks that the code is refused (400, "no longer taken") and never sent back on edit.

- **A timing flake in Collab's worker tests** (`hub_ports`, a delayed queue message) failed master's Ubuntu job once; its stopwatch now starts before the send.
- **A worker race** (from the T308 run): after a restart two jobs shared one temporary catalogue file and one stalled for 30 minutes. Fixed by a unique temporary name.

Release and deploy steps that stay yours

None of these is needed for the home-network tests.

- **Starfront server deploy** (`Deploy the Starfront server.cmd`): ships the hub API, the `/stack` pages and the front door. It runs `install.sh`, which now installs `python-multipart`.
- **Starfront desktop installer** (`packaging\build.ps1`) and **plugin release**, after T507 and T606 pass.
- **Collab worker and hub release**: the worker-release workflow, then sign the manifests with your key (item 1) and copy them into `server/downloads/`.
- **Collab's AWS stack** needs nothing for the hub. Its Terraform changed only in the worker-config name list; apply it whenever you next deploy.

Housekeeping

- **Unversioned files changed:** the suite guides `Astro/Rust/CLAUDE.md` and `AGENTS.md` now say contract 1.29.0 and describe the hub. `Running-the-servers-locally.md` points to the new runbook and `local_night.py`; it now lives in this repository beside its PDF, and the Astro-level copy is gone.
- **Your main Collab checkout** still holds a built Lambda tree from the last deploy, so the local-stack tests fail there. Run them in a worktree, or run `backend/scripts/clean.sh` when no deploy is pending.
- **Branches:** every task branch is merged. Starfront's `upstream` remote has a `cursor/collab-live-stack-23be` branch that this work did not create or touch.
- **Contract mentions in Starfront prose:** `README.md` and `server/stack/cloud.py` say "the contract (1.28.0)" about the delete routes. Still true of 1.29.0, so left alone.
- **Progress page:** https://claude.ai/artifact/WJummaRZgi5Z3v6asiT9MX (refreshed when anything lands).